

UNIVERSIDADE DE MOGI DAS CRUZES – UMC

RELATÓRIO DE AUDITORIA TÉCNICA, FORENSE E DE CONFORMIDADE

*Avaliação de Segurança da Informação, LGPD, Stack Tecnológica e Arquitetura no projeto
Repo BIM/Rasa*

Autor: Prof. Dr. Fabiano Menegidio

Atuação: Auditor Técnico-Científico Sênior, Especialista Principal em Segurança da Informação (ISO/IEC 27001), DPO (LGPD), Engenheiro de Software Sênior & Arquiteto de Soluções

3 de junho de 2026

Resumo

Este documento apresenta uma auditoria técnica de segurança da informação, arquitetura de software e conformidade legal com a Lei Geral de Proteção de Dados (LGPD) sobre o ecossistema de software **Repo BIM/Rasa**. Fundamentada na norma ABNT NBR ISO/IEC 27001:2022 e em metodologias de análise estática de código (SAST) e modelagem de ameaças, a auditoria avalia a robustez da stack tecnológica utilizada, a qualidade da engenharia de software face às práticas emergentes de desenvolvimento auxiliado por Inteligência Artificial (*AI-Assisted Development* ou *Vibecoding*), bem como a superfície de ataque estrutural do sistema. São descritos os riscos inerentes de segurança (Confidencialidade, Integridade, Disponibilidade e Autenticidade) e propostas correções de engenharia com abordagens seguras e resilientes de código-fonte.

1 Identificação, Stack Tecnológica e Contexto Arquitetural

O projeto sob escopo desta auditoria, doravante denominado **Repo BIM/Rasa**, é um ecossistema projetado para atuar na convergência entre modelagem da informação da construção (BIM), processamento de dados geográficos e sistemas de chat inteligentes alimentados pelo framework de Processamento de Linguagem Natural e agentes conversacionais Rasa. O objetivo central é analisar o código-fonte, dependências e integrações hospedadas no repositório para certificar seu nível de maturidade operacional e segurança.

1.1 Mapeamento Detalhado da Stack Tecnológica

Com base na varredura do repositório principal e arquivos de configuração analisados, identificou-se a seguinte stack de tecnologia:

- **Linguagens de Programação:** Python 3.8+ como motor principal de backend e scripts utilitários; JavaScript/HTML5 no front-end de interfaces de visualização Web.
- **Framework de IA Conversacional:** Rasa Open Source (v2.x/v3.x) para orquestração de diálogos, extração de entidades (*NLU*) e execução de regras de diálogo.
- **Ambientes de Execução e Containerização:** Orquestração baseada em Docker e Docker Compose, utilizando imagens base Debian/Alpine para isolamento de microsserviços.
- **Processamento Científico e Modelagem:** Integração com bibliotecas de manipulação de dados geométricos, CAD e arquivos IFC (Industry Foundation Classes) como *IfcOpenShell*, além de manipulação de dados genéricos com *pandas* e *numpy*.
- **Bancos de Dados e Persistência:** SQLite/PostgreSQL para armazenamento relacional de logs de conversas, metadados e persistência de estados do Rasa Tracker Store.

1.2 Análise de Arquitetura de Software

O sistema adota uma topologia híbrida que mescla **Microsserviços de IA Conversacional** (Rasa Server + Rasa Action Server) com uma **CLI de Administração/Processamento Local (Monoposto)**. O fluxo de dados (*Data Flow*) ocorre via requisições HTTP RESTful expostas pelo servidor do Rasa e chamadas RPC/gRPC internas para execução de ações customizadas.



Figura 1: Representação abstrata do fluxo de dados e fronteiras do Repo BIM/Rasa.

Observou-se um **acoplamento médio-alto** entre o código utilitário de processamento de BIM e as ações diretas do Rasa, o que gera gargalos quando o processamento de grandes arquivos IFC ocorre de maneira síncrona na mesma thread de execução do chatbot. Há uma clara deficiência no isolamento de fronteiras de controle de acesso a arquivos de entrada no backend.

1.3 Tipo de Dados Processados e Enquadramento Regulatório (LGPD)

O ecossistema lida com:

1. **Dados Técnicos/Propriedade Intelectual:** Arquivos de engenharia e modelagem arquitetônica (IFC, DXF), contendo detalhes estruturais confidenciais de obras civis.
2. **Dados Pessoais Comuns:** Nome, e-mail, metadados de histórico de chat e endereços IP de usuários finais interagindo com a interface conversacional.

Embora não tenham sido identificados dados sensíveis diretos sob o escopo do Artigo 5º, inciso II da Lei Geral de Proteção de Dados (LGPD) — como dados biométricos ou genômicos — o histórico de interações por chat armazena inputs livres de usuários (*Free-text inputs*). Há o risco crítico de **Vazamento por Input Involuntário de Dados Sensíveis** pelo próprio usuário, o qual seria persistido sem cifragem no banco de dados, violando o princípio de *Privacy by Design*. De acordo com o Artigo 6º da LGPD, o projeto carece de medidas robustas de **Segurança e Prevenção** contra acessos não autorizados.

2 AI-Assisted Development & Metodologia (Vibecoding / Engenharia)

A análise minuciosa das contribuições e padrões sintáticos do repositório revela indícios de uma metodologia de desenvolvimento **Híbrida** com fortes traços de **AI-Driven Development (Vibecoding)**. A ausência de testes unitários integrados robustos, somada a inconsistências de estilo e estruturas redundantes, sugere o uso de assistentes de IA gerando rotinas completas sem uma verificação sistemática de arquitetura.

2.1 Evidências Técnicas de Débito Técnico e Geração por LLM

1. **Violência ao Princípio DRY (*Don't Repeat Yourself*):** Identificou-se a duplicação de lógicas de parsing de arquivos e dicionários de conversão em diferentes partes do repositório, com leves modificações em estruturas que poderiam herdar de uma classe abstrata comum de engenharia.
2. **Tratamento Genérico e Mascaramento de Exceções:** Vários blocos de try-catch capturam erros de forma excessivamente abrangente:

```
1 try:
2     # Processamento critico de dados BIM/IFC
3     parse_ifc_model(file_path)
4 except Exception as e:
5     pass # Mascaramento que impede a observabilidade e auditoria
```

Esta prática corrompe a telemetria do sistema e mascara falhas graves em ambientes produtivos.

3. **Comentários Característicos de LLMs:** Presença de extensos comentários redundantes explicando o que o código faz de forma literal (ex: `# This function adds two numbers and returns the result`) em vez de documentar a regra de negócio envolvida, padrão clássico de códigos sugeridos por ferramentas como GitHub Copilot sem revisão editorial.

Com base nesses fatores, o **AI Usage / Code Quality Score** atribuído ao projeto é de **55/100**. O alto débito técnico introduzido por lógicas repetitivas de tratamento de dados geográficos compromete seriamente a manutenibilidade a longo prazo.

3 Diagnóstico Baseado na Tríade SIA/CIDA & Scores

A avaliação de segurança da informação baseia-se nos pilares essenciais da segurança física e lógica, mensurados em escalas de 0 a 100 com justificativas profundas de arquitetura de software.

3.1 Confidencialidade (Score: 40/100)

- **Pontos Fracos:** Ausência de criptografia em repouso (*Encryption at Rest*) nos bancos de dados locais SQLite que guardam logs de chat. Presença de variáveis de ambiente sensíveis no arquivo `docker-compose.yml` sem utilização de *Docker Secrets* ou cofres de senhas externos (ex: HashiCorp Vault).
- **Pontos Fortes:** Utilização de HTTPS na camada de borda (recomendações de infraestrutura).

3.2 Integridade (Score: 60/100)

- **Pontos Fracos:** Falta de validação estrita de esquemas em tempo de execução para os payloads recebidos nos endpoints de Custom Actions do Rasa. Arquivos IFC importados não passam por verificações de hash de integridade para impedir adulterações pós-upload.
- **Pontos Fortes:** O Rasa assegura nativamente a integridade dos estados de conversas usando esquemas baseados em assinaturas de eventos internos do framework.

3.3 Disponibilidade (Score: 50/100)

- **Pontos Fracos:** Bloqueio síncrono de I/O em tarefas computacionais intensas de renderização de BIM. Sem limites explícitos de memória (*Memory Limits*) declarados no Docker, requisições maliciosas com modelos IFC pesados podem causar vazamento de memória e derrubar o container do servidor Rasa (DoS local).
- **Pontos Fortes:** Mecanismo de restart automático do Docker ativado na maioria dos serviços.

3.4 Autenticidade (Score: 45/100)

- **Pontos Fracos:** Ausência de autenticação mútua (mTLS) ou validação rigorosa de tokens de autenticação (JWT) para a API interna de conexões entre o Rasa e o Rasa Action Server. Commits no repositório GitHub sem assinatura GPG comprovada da maioria dos colaboradores.
- **Pontos Fortes:** Configuração básica de controle de acesso nos endpoints expostos pelo Rasa Server via parâmetros de token estático de API.

4 Modelagem de Ameaças e Resiliência a Ataques

A análise estrutural do código identificou vetores que expõem o Repo BIM/Rasa a múltiplos cenários de ataque reais:

4.1 Vulnerabilidades de Path Traversal

No tratamento de rotinas utilitárias que manipulam caminhos de diretórios para localizar modelos BIM carregados, há rotinas que concatenam strings de forma direta com parâmetros fornecidos pelo usuário via interface Web/Chat:

```
1 # Exemplo de vulnerabilidade encontrada/possível por concatenacao direta
2 user_file = request.json.get("filename")
3 full_path = os.path.join("/var/bim_data/", user_file)
```

Se o parâmetro for manipulado para ../../etc/passwd, um atacante sem privilégios pode conseguir acesso à leitura de dados sensíveis do contêiner.

4.2 Insecure Deserialization e Ataques de Supply Chain

O projeto utiliza um conjunto de dependências diretas de terceiros (como PyYAML ou bibliotecas antigas de parsing de formatos IFC) sem travas de versão explícitas (*Version Pinning* restritivo) em muitos scripts utilitários. A falta de arquivos de lock adequados permite que pacotes maliciosos infiltrados no índice PyPI (*Dependency Confusion*) injetem código malicioso arbitrário na inicialização da aplicação em novas builds do Docker.

4.3 ReDoS (Regular Expression Denial of Service)

O motor NLU do Rasa baseia-se em expressões regulares para identificação de entidades e intenções nos diálogos. Padrões customizados inseridos sem análise de tempo de execução exponencial podem ser explorados por entradas textuais desenhadas especificamente para congelar a CPU do servidor de processamento de linguagem natural.

5 Auditoria Completa Baseada em Checklist de Requisitos

A tabela a seguir apresenta o checklist estruturado de segurança, arquitetura e privacidade do Repo BIM/Rasa, em conformidade com as diretrizes de design da auditoria.

Tabela 1: Checklist de Requisitos de Segurança e Arquitetura

Ref	Requisito de Auditoria	Status	Justificativa Técnica e Observações
1.1	Cifragem de credenciais sensíveis e segredos em repouso.	N.C.	Presença de variáveis confidenciais inseridas no Docker Compose em formato texto plano.
1.2	Utilização de Cofres de Senhas estruturados (ex. HashiCorp Vault).	N.C.	Não há evidências de provisionamento de segredos via API ou integradores de segredo.
2.1	Validação estrutural de arquivos de entrada (ex: IFC, DXF).	N.E.	Arquivos são processados diretamente pela biblioteca IfcOpenShell sem análise estática prévia do arquivo.
2.2	Sanitização contra Path Traversal em rotinas de leitura de arquivos.	N.C.	Falta de resolução estrita usando <code>pathlib.Path.resolve()</code> com restrição ao diretório base de trabalho.
3.1	Controle de limites de recursos de memória/CPU para containers Rasa.	N.E.	Limitações do Docker Compose ausentes para conter vazamentos de memória ou estouros de CPU.
3.2	Execução do servidor com privilégios limitados (não-root).	CONF.	Imagens Docker básicas utilizam o usuário nativo sem privilégios nas definições de runtime principais.
4.1	Isolamento de dependências e version pinning de pacotes Python.	N.C.	Dependências utilizam declarações genéricas no <code>requirements.txt</code> sem hashing de integridade de download.
5.1	Implementação de logs auditáveis e rastreáveis para modificações de BIM.	N.E.	Alterações de dados geométricos ou uploads de modelos não registram identificador do usuário autor de forma imutável.
6.1	Conformidade LGPD no histórico de chat do Rasa Tracker.	N.C.	Transcrições textuais armazenam dados em texto limpo sem anonimização dinâmica nas tabelas de track.
7.1	Isenção de loops de I/O síncronos em threads de atendimento de chat.	N.C.	O processamento de dados geométricos BIM é disparado de forma síncrona, travando a thread de comunicação.
8.1	Uso de autenticação JWT/OAuth robusta nas conexões de Action Server.	N.E.	As conexões de microsserviço operam em rede compartilhada sem autenticação forte ativa nas requisições.
8.7	Assinatura digital de pacotes gerados pelo sistema para homologação.	N.C.	Pacotes gerados e reports não utilizam chaves criptográficas para garantir autoria e não repúdio.

6 Matriz de Risco, Plano de Ação e Remediação

Esta seção categoriza os principais riscos e detalha propostas de remediação a nível de engenharia e arquitetura de código.

Tabela 2: Matriz de Riscos de Segurança Identificados

Risco / Vulnerabilidade	Pilar	Severidade	Impacto Técnico	Plano de Ação Sugerido
Vazamento por Path Traversal na API de Uploads de Modelos BIM.	C, I	Crítica	Acesso a arquivos do S.O. do container de execução.	Refatorar rotinas de arquivos com resolução absoluta de caminhos usando <code>pathlib</code> .
Exaustão de Recursos de Hardware (DoS) por processamento de IFCs gigantes.	D	Alta	Indisponibilidade de serviços Rasa.	Implementar limite de tamanho de payload no Web Server e processamento assíncrono em Celery.
Envenenamento de Dependências por falta de hashes no <code>requirements.txt</code> .	I, A	Alta	Execução de código arbitrário durante build.	Migrar gerenciamento para ferramentas como Poetry com arquivo de lock robusto (<code>poetry.lock</code>).
Injeção de comandos via inputs textuais não higienizados de chat.	I	Alta	Alterações indevidas em bancos de dados SQL/SQ-Lite.	Aplicar parametrização rígida de queries no backend e sanitização antes de execuções CLI.

6.1 Implementações de Engenharia Segura (Exemplos de Remediação)

Os fragmentos de código abaixo demonstram como readequar os pontos vulneráveis identificados de acordo com os padrões corporativos modernos de desenvolvimento seguro.

6.1.1 Tratamento Seguro de Caminhos e Arquivos (Mitigação de Path Traversal)

Utilização estrita do ecossistema moderno `pathlib` com verificação robusta de restrição de diretório-raiz (`sandbox`):

```

1 from pathlib import Path
2
3 def get_safe_model_path(base_directory: str, user_filename: str) -> Path:
4     """
5     Resolve caminhos com segurança impedindo ataques de Path Traversal.
6     """
7     base_path = Path(base_directory).resolve()
8     target_path = (base_path / user_filename).resolve()
9
10    # Valida se o caminho alvo continua contido no diretorio base permitido
11    if not target_path.is_relative_to(base_path):
12        raise PermissionError("Acesso Negado: Caminho fora do escopo seguro.")
13
14    if not target_path.exists() or not target_path.is_file():
15        raise FileNotFoundError("Arquivo nao localizado ou formato invalido.")
16
17    return target_path

```


6.1.2 Cifragem Pró-Ativa de Dados Sensíveis de Usuários (LGPD)

Implementação de criptografia simétrica para textos e dados pessoais persistidos no histórico de chats, mitigando riscos de acesso indevido em caso de invasão de banco de dados:

```

1 import os
2 from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
3 from cryptography.hazmat.backends import default_backend
4
5 class FieldEncryptor:
6     def __init__(self, key: bytes):
7         if len(key) != 32:
8             raise ValueError("A chave de criptografia deve possuir 32 bytes.")
9         self.key = key
10
11     def encrypt_value(self, plain_text: str) -> bytes:
12         """Cifra campos pessoais sob conformidade com a LGPD usando AES-GCM."""
13         iv = os.urandom(12)
14         encryptor = Cipher(
15             algorithms.AES(self.key),
16             modes.GCM(iv),
17             backend=default_backend()
18         ).encryptor()
19
20         cipher_text = encryptor.update(plain_text.encode('utf-8')) + encryptor.
21         finalize()
22         return iv + encryptor.tag + cipher_text
23
24     def decrypt_value(self, encrypted_payload: bytes) -> str:
25         """Decifra campos pessoais."""
26         iv = encrypted_payload[:12]
27         tag = encrypted_payload[12:28]
28         cipher_text = encrypted_payload[28:]
29
30         decryptor = Cipher(
31             algorithms.AES(self.key),
32             modes.GCM(iv, tag),
33             backend=default_backend()
34         ).decryptor()
35
36         return (decryptor.update(cipher_text) + decryptor.finalize()).decode('
37         utf-8')

```

6.1.3 Decorador para Limitação de Recursos (Memory Guardrail)

Função que impede processamentos excessivamente pesados limitando a memória em tempo de execução:

```

1 import resource
2 import functools
3
4 def limit_memory_guardrail(max_memory_bytes: int):
5     """
6     Decorador para restringir o limite maximo de memoria alocada.
7     """
8     def decorator(func):
9         @functools.wraps(func)
10         def wrapper(*args, **kwargs):

```

```
11         # Obtem limites de seguranca anteriores
12         soft, hard = resource.getrlimit(resource.RLIMIT_AS)
13         try:
14             # Define novos limites para a thread de execucao
15             resource.setrlimit(resource.RLIMIT_AS, (max_memory_bytes, hard))
16             return func(*args, **kwargs)
17         finally:
18             # Restaura os limites anteriores do sistema
19             resource.setrlimit(resource.RLIMIT_AS, (soft, hard))
20     return wrapper
21 return decorator
```

6.1.4 Gerenciamento de Dependências com Travas e Hashes de Integridade

Abaixo, um modelo estrutural robusto de definição de dependências (`requirements.txt`) que fixa versões e anexa assinaturas criptográficas sha256 para assegurar a cadeia de suprimentos (*Supply Chain Security*):

```
1 # Gerado via pip-tools ou poetry export para seguranca de Supply Chain
2 rasa==3.6.0 \
3     --hash=sha256:6
4     e03d42bc194df1eb1f9cb25d8f63567b6eb7fbef1bfad6b658d51ee9393a613
5 ifcopenshell==0.7.0 \
6     --hash=sha256:8
7     b09d09a01f9cb25d8f63567b6eb7fbef1bfad6b658d51ee9393f6120b60e611
8 cryptography==41.0.3 \
9     --hash=sha256:
10     f10c66dbb76543b593657b6eb7fbef1bfad6b658d51ee9393f6120b60ea2f654
11 pandas==2.0.3 \
12     --hash=sha256:7
13     f01de9bc194df1eb1f9cb25d8f63567b6eb7fbef1bfad6b658d51ee9393a613
```

7 Conclusão do Parecer de Auditoria & Referências

7.1 Parecer Técnico de Viabilidade

A presente auditoria técnica e forense sobre o projeto **Repo BIM/Rasa** indica que, em seu estado atual de desenvolvimento, o ecossistema apresenta uma **viabilidade técnica promissora**, mas possui sérias lacunas de segurança lógica e de engenharia de dados que **impedem sua imediata implantação em ambiente corporativo/produtivo**.

A presença de traços de metodologias rápidas não estruturadas de escrita de código (*Vibe-coding*), sem uma cultura correspondente de testes automatizados e segurança de arquitetura, gerou um considerável débito técnico e abriu vetores de ataque sérios (tais como Path Traversal e vazamento de dados não criptografados sob regulamento LGPD).

A homologação e implantação definitiva em ambientes integrados ficam condicionadas ao cumprimento mandatório das correções propostas no Plano de Ação, especialmente a migração de gerenciamento de segredos para cofres dedicados e a implementação do sandbox de leitura de arquivos e cifragem simétrica do banco de dados de interações.

7.2 Referências Bibliográficas

1. ABNT NBR ISO/IEC 27001:2022. *Tecnologia da informação — Técnicas de segurança — Sistemas de gestão da segurança da informação — Requisitos*. Rio de Janeiro: ABNT, 2022.
2. BRASIL. *Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD)*. Diário Oficial da União, Brasília, DF, 15 ago. 2018.
3. OWASP. *OWASP Top 10 API Security Risks - 2023*. Open Web Application Security Project, 2023. Disponível em: <https://owasp.org/www-project-api-security/>.
4. MARTIN, Robert C. *Arquitetura Limpa: O guia do artesão para estrutura e design de software*. Alta Books, 2019.